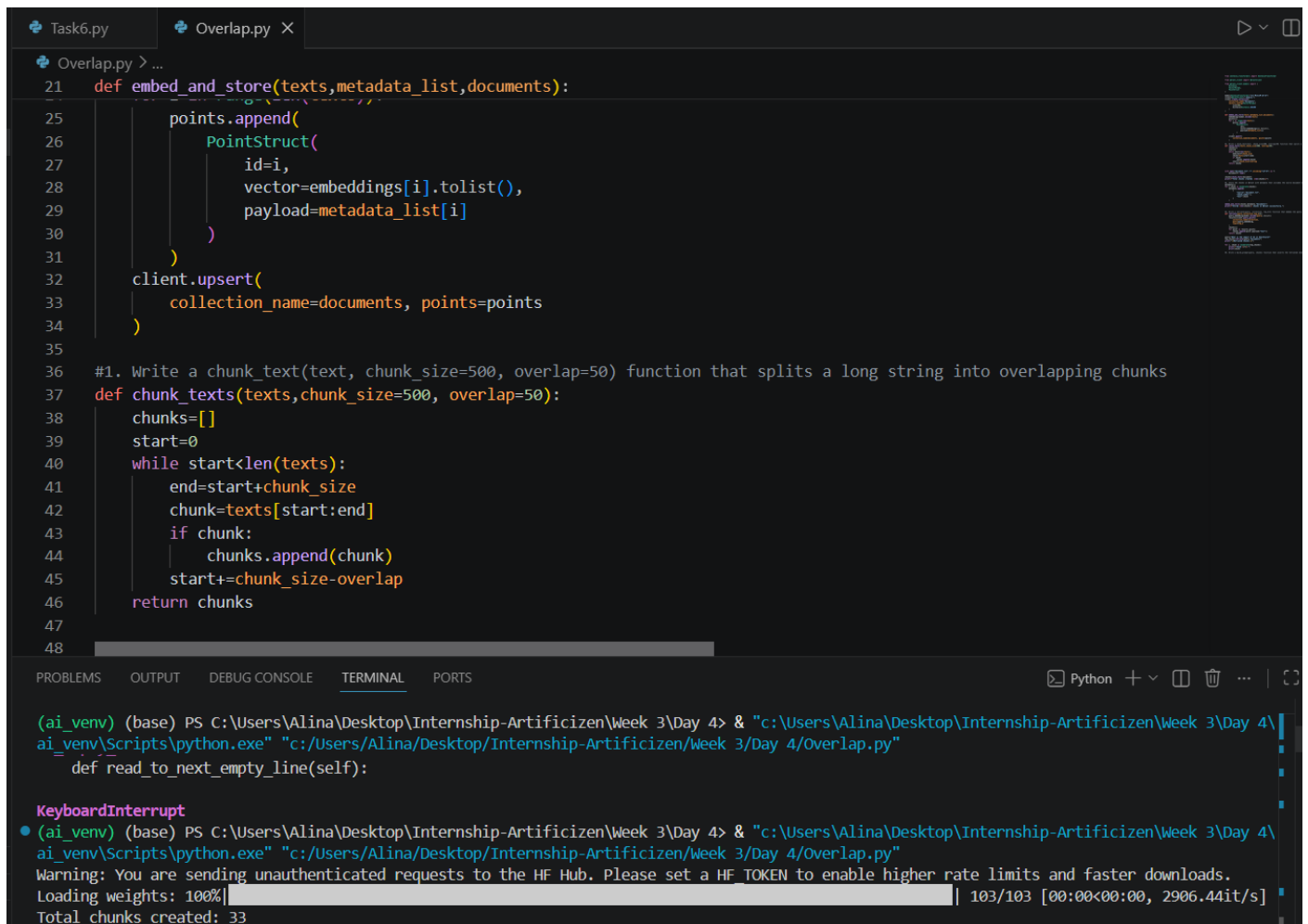


Practice Questions

1. Write a `chunk_text(text, chunk_size=500, overlap=50)` function that splits a long string into overlapping chunks. Print the number of chunks produced from a 3,000-word document.



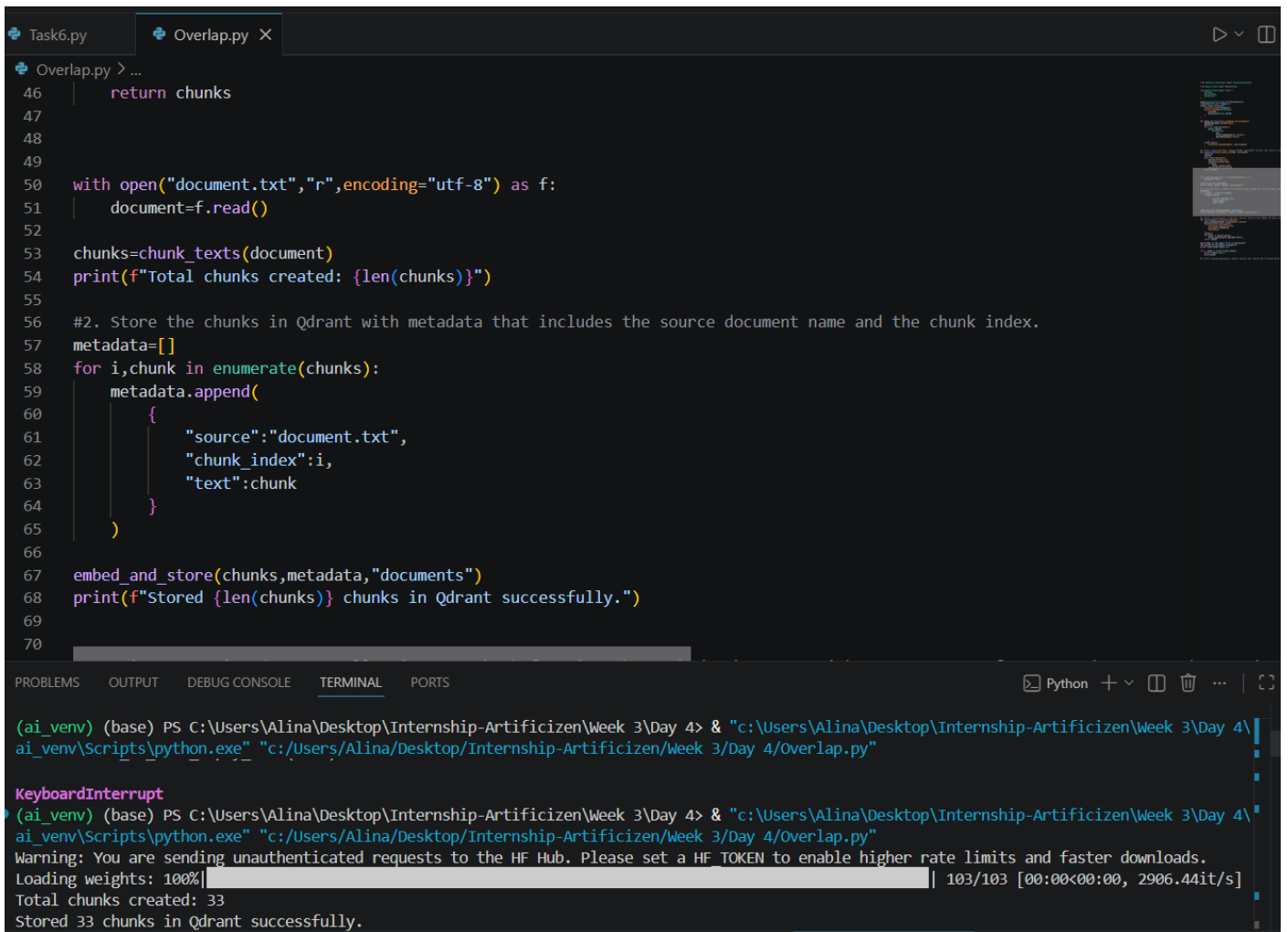
```
Task6.py Overlap.py X
Overlap.py > ...
21 def embed_and_store(texts, metadata_list, documents):
22     points.append(
23         PointStruct(
24             id=i,
25             vector=embeddings[i].tolist(),
26             payload=metadata_list[i]
27         )
28     )
29     client.upsert(
30         collection_name=documents, points=points
31     )
32
33 #1. Write a chunk_text(text, chunk_size=500, overlap=50) function that splits a long string into overlapping chunks
34 def chunk_text(texts, chunk_size=500, overlap=50):
35     chunks=[]
36     start=0
37     while start<len(texts):
38         end=start+chunk_size
39         chunk=texts[start:end]
40         if chunk:
41             chunks.append(chunk)
42         start+=chunk_size-overlap
43     return chunks
44
45
46
47
48
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + -

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/Overlap.py"
def read_to_next_empty_line(self):

KeyboardInterrupt
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/Overlap.py"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2906.44it/s]
Total chunks created: 33
```

2. Load a plain-text or PDF file, chunk it, embed all chunks with sentence-transformers, and store them in Qdrant with source filename and chunk index as metadata.



The image shows a Visual Studio Code editor window with two tabs: 'Task6.py' and 'Overlap.py'. The 'Overlap.py' tab is active, displaying a Python script. The script reads a document from 'document.txt', chunks it, and stores the chunks in Qdrant with metadata. The terminal at the bottom shows the execution of the script, which successfully stores 33 chunks.

```
Overlap.py > ...
46 |     return chunks
47 |
48 |
49 |
50 | with open("document.txt","r",encoding="utf-8") as f:
51 |     document=f.read()
52 |
53 | chunks=chunk_texts(document)
54 | print(f"Total chunks created: {len(chunks)}")
55 |
56 | #2. Store the chunks in Qdrant with metadata that includes the source document name and the chunk index.
57 | metadata=[]
58 | for i,chunk in enumerate(chunks):
59 |     metadata.append(
60 |         {
61 |             "source":"document.txt",
62 |             "chunk_index":i,
63 |             "text":chunk
64 |         }
65 |     )
66 |
67 | embed_and_store(chunks,metadata,"documents")
68 | print(f"Stored {len(chunks)} chunks in Qdrant successfully.")
69 |
70 |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [] [] [] [] [] []

```
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/Overlap.py"

KeyboardInterrupt
(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/Overlap.py"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2906.44it/s]
Total chunks created: 33
Stored 33 chunks in Qdrant successfully.
```

3. Write a retrieve(query, collection, top_k=3) function that embeds the query with sentence-transformers and returns the top-k chunks from Qdrant.

```
Task6.py Overlap.py X
Overlap.py > ...
66
67 embed_and_store(chunks,metadata,"documents")
68 print(f"Stored {len(chunks)} chunks in Qdrant successfully.")
69
70 #3. Write a retrieve(query, collection, top_k=3) function that embeds the query with sentence-transformers and returns the top k results
71
72 def retrieve(query,collection,top_k=3):
73     query_embedding=model.encode(query).tolist()
74     results=client.query_points(
75         collection_name=collection,
76         query=query_embedding,
77         limit=top_k
78     )
79     return results
80
81 if __name__ == '__main__':
82     chunks=create_chunks()
83     embed_and_store(chunks,metadata,"documents")
84     query="What is the history of artificial intelligence?"
85     results=retrieve(query,"documents",top_k=3)
86     print_results(results)
```

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/Overlap.py"

Total chunks created: 33
Stored 33 chunks in Qdrant successfully.

Retrieved Chunks:

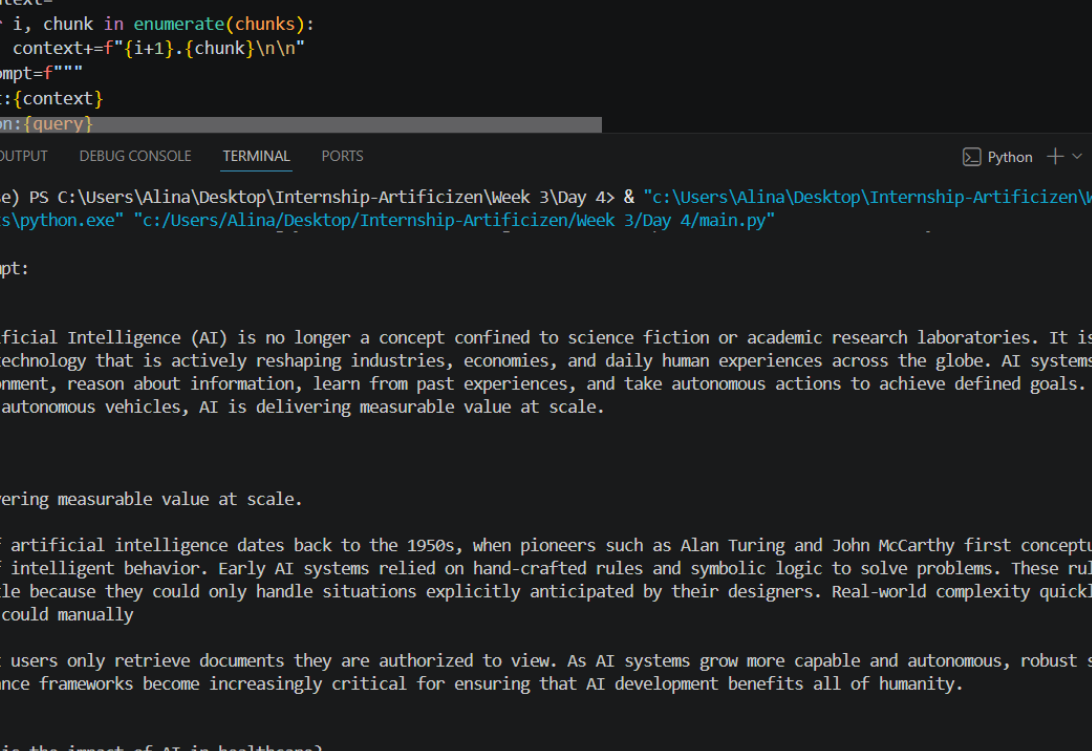
Chunk 1:
Artificial Intelligence (AI) is no longer a concept confined to science fiction or academic research laboratories. It is a practical, deployable technology that is actively reshaping industries, economies, and daily human experiences across the globe. AI systems can perceive their environment, reason about information, learn from past experiences, and take autonomous actions to achieve defined goals. From medical diagnosis to autonomous vehicles, AI is delivering measurable value at scale.

The
Chunk 2:
AI is delivering measurable value at scale.

The history of artificial intelligence dates back to the 1950s, when pioneers such as Alan Turing and John McCarthy first conceptualized machines capable of intelligent behavior. Early AI systems relied on hand-crafted rules and symbolic logic to solve problems. These rule-based systems were brittle because they could only handle situations explicitly anticipated by their designers. Real-world complexity quickly exceeded what any human could manually

Chunk 3:
ensure that users only retrieve documents they are authorized to view. As AI systems grow more capable and autonomous, robust safety research and governance frameworks become increasingly critical for ensuring that AI development benefits all of humanity.

4. Write a `build_prompt(query, chunks)` function that inserts the retrieved chunks as numbered context items and appends: "Answer using only the context above. If the answer is not in the context, say: I don't know."



```
main.py > ...
96 #4. Write a build_prompt(query, chunks) function that inserts the retrieved chunks as numbered context items and appends
97 def build_prompt(query, chunks):
98     context=""
99     for i, chunk in enumerate(chunks):
100         context+=f"{i+1}. {chunk}\n\n"
101     prompt=f"""
102     context:{context}
103     Question:{query}

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/main.py"

Generated Prompt:

context:1.Artificial Intelligence (AI) is no longer a concept confined to science fiction or academic research laboratories. It is a practical , deployable technology that is actively reshaping industries, economies, and daily human experiences across the globe. AI systems can perceive their environment, reason about information, learn from past experiences, and take autonomous actions to achieve defined goals. From medical diagnosis to autonomous vehicles, AI is delivering measurable value at scale.

The

2. AI is delivering measurable value at scale.

The history of artificial intelligence dates back to the 1950s, when pioneers such as Alan Turing and John McCarthy first conceptualized machines capable of intelligent behavior. Early AI systems relied on hand-crafted rules and symbolic logic to solve problems. These rule-based systems were brittle because they could only handle situations explicitly anticipated by their designers. Real-world complexity quickly exceeded what any human could manually

3. ensure that users only retrieve documents they are authorized to view. As AI systems grow more capable and autonomous, robust safety research and governance frameworks become increasingly critical for ensuring that AI development benefits all of humanity.

Question:What is the impact of AI in healthcare?
Answer using only the context above.

If the answer is not in the context, say:
I don't know.

- Wire everything together: `retrieve()` → `build_prompt()` → `ask()` (using Groq). Ask three questions — two with answers in the document and one without. Verify the model says “I don’t know” for the third.

```
main.py > ask
117 def ask(prompt):
120     messages=[
121         {
122             "role": "user",
123             "content": prompt
124         }
125     ]
126 )
127     return response.choices[0].message.content
128
129 questions = [
130     "What is RAG?",
131     "Why is overlap important in chunking?",
132     "Who is the CEO of Microsoft?"
133 ]
134 for question in questions:
135     chunks=retrieve(question,"documents")
136     prompt=build_prompt(question,chunks)
137     answer=ask(prompt)
138     print(f"\nQuestion: {question}")
139     print(f"Answer: {answer}")
```

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/main.py"

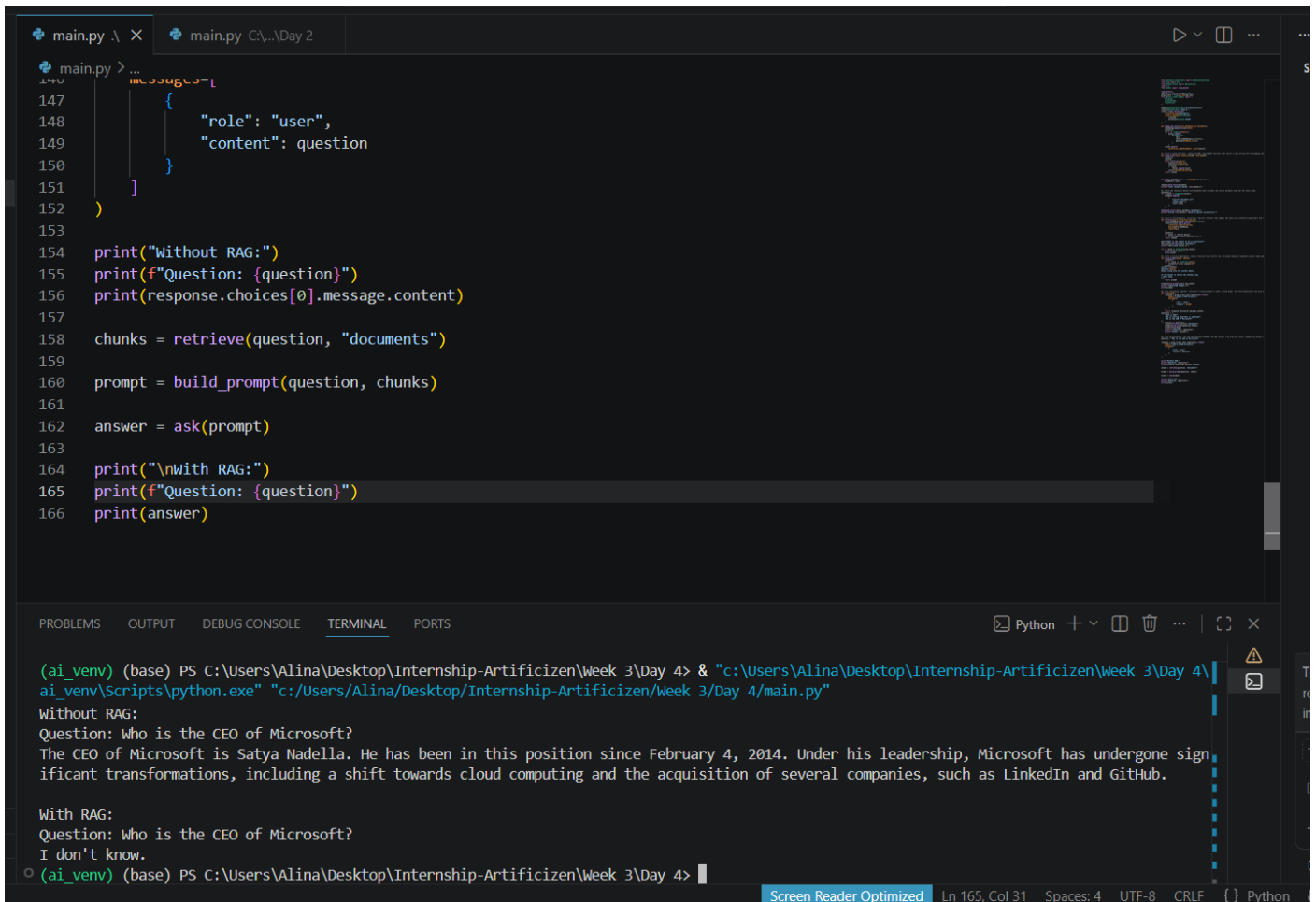
If the answer is not in the context, say:
I don't know.

Question: What is RAG?
Answer: RAG stands for Retrieval-Augmented Generation, an architectural pattern that combines the strengths of retrieval systems and generative language models.

Question: Why is overlap important in chunking?
Answer: I don't know.

Question: Who is the CEO of Microsoft?
Answer: I don't know.

6. Test hallucination: run the same question WITHOUT the RAG context (raw Groq call only). Compare the answer to the RAG answer. Write a short observation on which is more grounded and why.



```
main.py \ X main.py C:\...\Day 2
main.py > ...
147     messages=[
148         {
149             "role": "user",
150             "content": question
151         }
152     ]
153
154     print("Without RAG:")
155     print(f"Question: {question}")
156     print(response.choices[0].message.content)
157
158     chunks = retrieve(question, "documents")
159
160     prompt = build_prompt(question, chunks)
161
162     answer = ask(prompt)
163
164     print("\nWith RAG:")
165     print(f"Question: {question}")
166     print(answer)
```

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4> & "c:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4\ai_venv\Scripts\python.exe" "c:/Users/Alina/Desktop/Internship-Artificizen/Week 3/Day 4/main.py"

Without RAG:
Question: Who is the CEO of Microsoft?
The CEO of Microsoft is Satya Nadella. He has been in this position since February 4, 2014. Under his leadership, Microsoft has undergone significant transformations, including a shift towards cloud computing and the acquisition of several companies, such as LinkedIn and GitHub.

With RAG:
Question: Who is the CEO of Microsoft?
I don't know.

(ai_venv) (base) PS C:\Users\Alina\Desktop\Internship-Artificizen\Week 3\Day 4>

Screen Reader Optimized Ln 165, Col 31 Spaces: 4 UTF-8 CRLF () Python